

Modèles et Algorithmes pour les Graphes (MAG)

L3 Informatique/MIAGE

2023–2024

TD6-bis : Arbre couvrant de poids minimal : algorithme de Kruskal

Algorithm 1 ACM_generique($G(V, E), w$)

Require: Un graphe $G = (V, E)$ avec des poids $w_e, e \in E$.

Ensure: Un ACM défini par les arêtes de l'ensemble X .

```
1:  $X \leftarrow \emptyset$ 
2: while  $X$  ne forme pas un arbre couvrant do
3:   trouver pour  $X$  une arête sûre  $(u, v)$ 
4:    $X \leftarrow X \cup \{(u, v)\}$ 
5: end while
6: return  $X$ 
```

Algorithme de Kruskal et structures de données pour les ensembles disjoints

Algorithm 2 ACM_Kruskal($G(V, E), w$)

Require: Un graphe $G = (V, E)$ avec des poids $w_e, e \in E$.

Ensure: Un ACM défini par les arêtes de l'ensemble X .

```
1: for all  $u \in V$  do
2:   makeset( $u$ )
3: end for
4:  $X \leftarrow \emptyset$ 
5: trier les arêtes  $E$  par leurs poids
6: for all  $(u, v) \in E$ , dans l'ordre croissant des poids do
7:   if find( $u$ )  $\neq$  find( $v$ ) then
8:      $X \leftarrow X \cup \{(u, v)\}$ 
9:     union( $u, v$ )
10:  end if
11: end for
12: return  $X$ 
```

Algorithm 3 procedure makeset(x)

Ensure: crée un nouvel ensemble dont le seul élément (et donc le représentant) est x .

```
 $\pi(x) \leftarrow x$ 
 $rank(x) \leftarrow 0$ 
```

Algorithm 4 function find(x)

Ensure: retourne un pointeur vers le représentant de l'ensemble (unique) contenant x .

```
    while  $x \neq \pi(x)$  do
         $x \leftarrow \pi(x)$ 
    end while
```

return x

Algorithm 5 function find_bis(x)

Ensure: effectue une compression du chemin lors de l'opération find

```
    if  $x \neq \pi(x)$  then
         $\pi(x) \leftarrow \text{find}(\pi(x))$ 
    end if
```

return $\pi(x)$

Algorithm 6 procedure union(x, y)

Require: Deux ensembles dynamiques S_x et S_y représentés par leurs racines x et y . S_x et S_y sont supposés disjoints avant l'opération.

Ensure: Réunit les ensembles dynamiques S_x et S_y dans $S_x \cup S_y$. On fait pointer la racine du moindre rang sur celle de rang supérieur au moment de l'union. Comme les ensembles de la collection sont obligatoirement disjoints, on supprime les ensembles S_x et S_y .

```
     $r_x \leftarrow \text{find}(x)$ 
     $r_y \leftarrow \text{find}(y)$ 
    if  $r_x = r_y$  then
        return
    end if
    if rank( $r_x$ ) > rank( $r_y$ ) then
         $\pi(r_y) \leftarrow r_x$ 
    else
         $\pi(r_x) \leftarrow r_y$ 
        if rank( $r_x$ ) = rank( $r_y$ ) then
            rank( $r_y$ )  $\leftarrow$  rank( $r_y$ ) + 1
        end if
    end if
```

Exercice 1. Des chemins forestiers

Les Eaux et Forêts ont décidé d'abattre 5 bosquets situés dans une forêt. Les distances mutuelles entre les bosquets sont données (en km) par la matrice suivante.

	a	b	c	d	e
a		11	1	2	6
b	11		7	5	3
c	1	7		4	10
d	2	5	4		8
e	6	3	10	8	

Pour mener à bien cette exploitation il est nécessaire de tracer un réseau de chemins de coût de construction minimum qui permette de circuler entre tous les bosquets. Les coûts de construction de ces chemins sont proportionnels à leurs longueurs.

1. Énumérer les algorithmes vus en cours pouvant résoudre ce problème.
2. Appliquer l'algorithme de votre choix sur le graphe représenté par la matrice proposée dans l'énoncé. Le calcul à chaque itération sera clairement indiqué.

Exercice 2. À cause d'une dégradation de la communication lors du cours en distance en 2020 sur les arbres couvrants minimaux une étudiante de L3 Info / MIAE de l'ISTIC n'a pas pu suivre les explications données par l'enseignant concernant les structures de données pour les ensembles disjoints. Elle ne s'est pas découragée et a décidé de concevoir elle-même les codes des procédures `makeset`, `find`, `union`. Sa proposition est ci-dessous présentée.

Algorithm 7 procedure `makeset(SET, V)`

Ensure: crée un tableau `SET` tel que $SET[v] = v, \forall v \in V$ où V est l'ensemble des sommets du graphe

for all $v \in V$ **do**

$SET[v] \leftarrow v$

end for

Algorithm 8 function `find(SET, x)`

Ensure: retourne un pointeur vers le représentant de l'ensemble (unique) contenant x .

return $SET[x]$

Algorithm 9 `procedure union( $SET, x, y$ )`

Require: Deux ensembles S_x et S_y représentés par leurs racines $SET[x]$ et $SET[y]$.

Ensure: Réunit les ensembles S_x et S_y dans $S_x \cup S_y$. On fait pointer la racine de y sur celle de la racine de x .

```
for all  $v \in V$  do
  if  $SET[v] = SET[y]$  then
     $SET[v] \leftarrow SET[x]$ 
  end if
end for
```

1. Exécuter les programmes `makeset`, `find`, `union` sur le graphe des cinq bosquets. Afficher l'évolution du tableau `SET`.
2. Quelle serait la complexité de l'algorithme de Kruskal si on utilisait ces codes ?
3. Démontrer la complexité de l'algorithme original de Kruskal (**Algorithme 2**).
4. Comparer et analyser les deux complexités calculées.

Exercice 3. Peut-on adapter l'algorithme de Kruskal pour trouver un arbre couvrant de poids maximal ?

Exercice 4. Montrer que pour tout graphe, lorsque ses poids sont distincts deux à deux, la solution au problème de l'ACM est unique. En déduire le nombre de solutions pour le graphe de l'énoncé.

Exercice 5. Montrer que si une arête $e = (u, v)$ appartient à un arbre couvrant minimal A , c'est une arête minimale traversant une coupure du graphe.